

Why Computer Prefers 2's Complement in Signed Data Representation

Numbers	Sign Magnitude signed data	1's Complement signed data	2's Complement signed data
3	0 1 1	0 1 1	0 1 1
2	0 1 0	0 1 0	0 1 0
1	0 0 1	0 0 1	0 0 1
0	0 0 0	0 0 0	0 0 0
-1	1 0 1	1 1 0	1 1 1
-2	1 1 0	1 0 1	1 1 0
-3	1 1 1	1 0 0	1 0 1
-4	X	X	1 0 0

100 is not used,
100 indicates (-0)

111 is not used,
111 indicates (-0)

1's comp 2's comp
100 → 011 + 1
 = 100

→ In 2's Complement method, we don't waste any digit, so we use 2's complement method in computer for data representation.

Signed Data Representation and Range of Signed Numbers in Computer

→ For n bits

Range $+(2^{n-1}-1)$ to $-(2^{n-1})$

→ For 3 bits,

+3 to -4

→ For 4 bits

+7 to -8

→ For 8 bits

+255 to -256

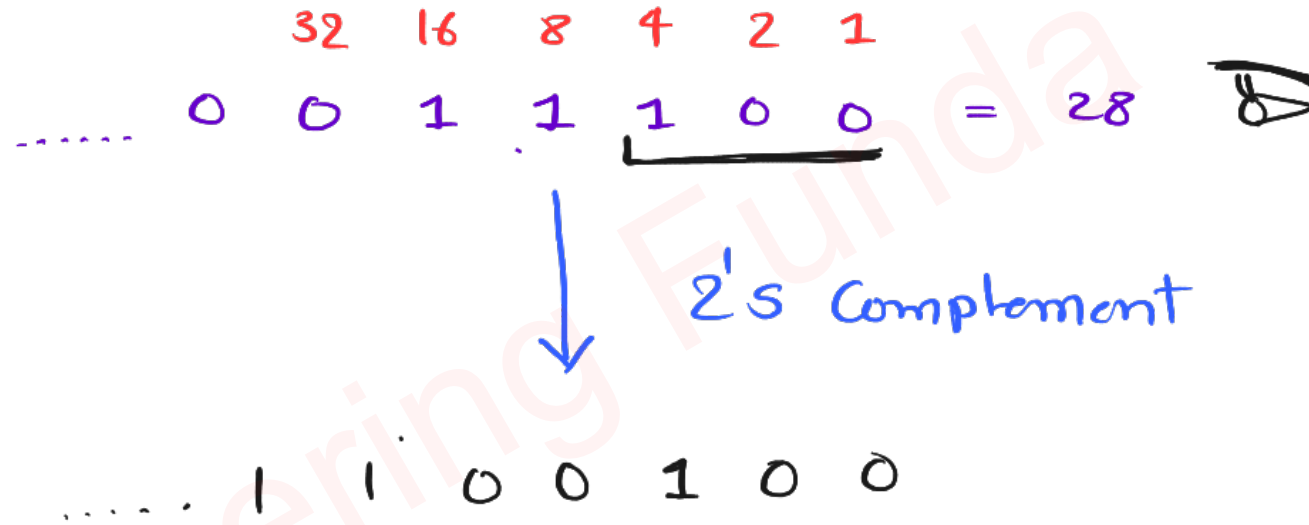
→ 110011 → 2^1 Comp. $\overline{001101}$
32 16 8 4 2 1
= -13

→ 1111 00100 → 2^1 Comp. $\overline{000011100}$
16 8 4 2 1
= -28
-ve extension by 1's
+ve extension by 0's

Examples on Number Representation

□ **GATE 2019 CS** – In 16 bit 2's Complement representation, the decimal number -28 is:

- A. 1111 1111 0001 1100
- B. 0000 0000 1110 0100
- ✓ C. 1111 1111 1110 0100
- D. 1000 0000 1110 0100



❑ **GATE 2021 CS** – If the numerical value of a 2 byte unsigned integer on a little endian computer is 255 more than that on a big endian computer, which of the following choices represents the unsigned integer on a little endian computer?

✓ **A. 0x6665**

~~B. 0x0001~~

~~C. 0x4243~~

✓ **D. 0x0100**

$$\begin{array}{r}
 \begin{array}{cc}
 & 16 \\
 5 & 5
 \end{array} \\
 \underline{66} & \underline{65} & \text{L.E.} \\
 65 & 66 & \text{B.E.} \\
 \hline
 00 & FF \\
 & = 255
 \end{array}$$

$$\begin{array}{r}
 \begin{array}{cc}
 00 & 01 \\
 \hline
 01 & 00
 \end{array} & \begin{array}{l} \text{L.E.} \\ \text{B.E.} \end{array} \\
 - & \\
 \hline
 -X
 \end{array}$$

$$\begin{array}{r}
 \begin{array}{cc}
 & 15 & 16 \\
 42 & 43 & \text{L.E.} \\
 \hline
 43 & 42 & \text{B.E.} \\
 -X
 \end{array}
 \end{array}$$

$$\begin{array}{r}
 \begin{array}{cc}
 01 & 00 \\
 \hline
 00 & 01
 \end{array} & \begin{array}{l} \text{L.E.} \\ \text{B.E.} \end{array} \\
 \hline
 00 & FF \\
 & = 255
 \end{array}$$

❑ **GATE 2022 CS** – Let R1 and R2 be two 4 bits registers that stores numbers in 2's complement form. For the operation $R1 + R2$, which one of the following values of R1 and R2 gives an arithmetic overflow?

- ☒ A. R1 = 1011 and R2 = 1110
- ☒ B. R1 = 1100 and R2 = 1010
- ☒ C. R1 = 0011 and R2 = 0100
- ☒ D. R1 = 1001 and R2 = 1111

$$\begin{array}{r} \textcircled{1}\textcircled{0}1 \\ 1011 \\ 1110 \\ \hline 1001 \\ 1 \oplus 1 = 0 \end{array}$$

$$\begin{array}{r} \textcircled{1}\textcircled{0} \\ 1100 \\ 1010 \\ \hline 0110 \\ 1 \oplus 0 = 1 \end{array}$$

$$\begin{array}{r} \textcircled{0}\textcircled{0} \\ 0011 \\ 0100 \\ \hline 0111 \\ 0 \oplus 0 = 0 \end{array}$$

$$\begin{array}{r} \textcircled{1}\textcircled{1}11 \\ 1001 \\ 1111 \\ \hline 1000 \\ 1 \oplus 1 = 0 \end{array}$$

Examples on Number Representation

❑ **GATE 2013 CS** – The smallest integer that can be represented by 8 bits number in 2's complement form is

- A. -256
- ✓ B. -128
- C. -127
- D. 0

→ For n bits number range is

$$\begin{array}{ccc} + (2^{n-1} - 1) & \text{to} & - (2^{n-1}) \\ \hline \text{highest} & & \text{Lowest} \end{array}$$

$$= + (2^{8-1} - 1)$$

$$= +127$$

$$= - 2^{8-1}$$

$$= - 2^7$$

$$= -128$$

□ **GATE 2010 CS** – P is a signed Integer. The 2's complement representation of P is (F87B)₁₆. The 2's complement representation of 8xP is

- ✓ A. (C3D8)₁₆
- B. (187B)₁₆
- C. (F878)₁₆
- D. (987B)₁₆

$$P = F87B = \boxed{1111} 1000 0111 1011$$

↑
Sign



$$\begin{aligned} & 8 \times P \\ &= 2^3 \times P \end{aligned}$$



$$\boxed{11}00 0011 1101 \underline{1000}$$

↑
Sign

$$(C3D8)_{16}$$

Normalization of Floating Point Number

$$1) (01101.1011)_2 = (-1)^{\textcircled{0} \leftarrow \text{sign}} \times 1.\underbrace{1011011}_{\text{Mantissa}} \times 2^{\textcircled{3} \leftarrow \text{Exponent}}$$

$$2) (11.001110)_2 = (-1)^{\textcircled{0} \leftarrow \text{sign}} \times 1.\underbrace{100111}_{\text{Mantissa}} \times 2^{\textcircled{1} \leftarrow \text{Exponent}}$$

$$3) (0.0001011)_2 = (-1)^{\textcircled{0} \leftarrow \text{sign}} \times 1.\underbrace{011}_{\text{Mantissa}} \times 2^{\textcircled{-4} \leftarrow \text{Exponent}}$$

$$4) (-1101.0011)_2 = (-1)^{\textcircled{1} \leftarrow \text{sign}} \times 1.\underbrace{1010011}_{\text{Mantissa}} \times 2^{\textcircled{3} \leftarrow \text{Exponent}}$$

IEEE 754 32bits Single Precision Floating Point Representation



$$\rightarrow B.E. = E + 127$$

$$\rightarrow (76.125)_{10}$$

2	76	0
2	38	0
2	19	1
2	9	1
2	4	0
2	2	0
	1	

$$\begin{array}{lcl} 0.125 \times 2 & = & 0.25 & 0 \\ 0.25 \times 2 & = & 0.5 & 0 \\ 0.5 \times 2 & = & 1.0 & 1 \end{array}$$

$$(76.125)_{10} = (1001100.001)_2$$

$$= (-1)^{\text{sign}} \times 1.001100001 \times 2^{\text{Exponent}}$$

Mantissa

Answer

	128 64 32 16 8 4 2 1	
0	1 0 0 0 0 1 0 1	0 0 1 1 0 0 0 0 1 0
S	B.E = 127 + 6 = 133	m (23 bits).

IEEE 754 64bits Double Precision Floating Point Representation



$$\hookrightarrow B.E = E + 1023$$

$$(320.0625)_{10}$$

2	320	0
2	160	0
2	80	0
2	40	0
2	20	0
2	10	0
2	5	1
2	2	0
	1	

$$\begin{array}{rcll} 0.0625 \times 2 & = & \underline{0.125} & 0 \\ 0.125 \times 2 & = & \underline{0.25} & 0 \\ 0.25 \times 2 & = & \underline{0.5} & 0 \\ 0.5 \times 2 & = & \underline{1.0} & 1 \end{array} \quad \downarrow$$

$$(320.0625)_{10} = (101000000.0001)_2$$

$$= (-1)^{\text{Sign}} \times 1.\underbrace{010000000001}_{\text{Mantissa}} \times 2^{\text{Exponent}}$$

Answer

$$\begin{array}{r}
 0 \\
 \hline
 5
 \end{array}
 \begin{array}{cccccccccccc}
 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & \\
 \hline
 \text{Bias Exponent} \\
 = E + 1023 \\
 = 1031
 \end{array}
 \begin{array}{cccccccccccccccc}
 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & \dots
 \end{array}$$

Extreme Cases of Floating Point Representation

Normal Number

Sign	Bias Exponent	Mantissa
1 Bit	8 bits	23 bits

$S = 0 \rightarrow +ve$
 $S = 1 \rightarrow -ve$

$255 < B.E. < 0$

$\uparrow$
 Anything.

1) Not a Number (Overflow) (NaN)

Sign	Bias Exponent	Mantissa
1 Bit	8 bits	23 bits

$S = 0 \rightarrow +ve$
 $S = 1 \rightarrow -ve$

$B.E. = 255$
 $= 11111111$

$\uparrow$
 Anything.

2) Infinity (Overflow)

Sign	Bias Exponent	Mantissa
1 Bit	8 bits	23 bits

$S = 0 \rightarrow +ve$
 $S = 1 \rightarrow -ve$

$B.E. = 255$
 $= 11111111$

$m = 00 \dots 00$

3) Denormal Number { Underflow }

Sign	Bias Exponent	Mantissa
1 Bit	8 bits	23 bits

$S = 0 \rightarrow +ve$
 $S = 1 \rightarrow -ve$

$Bias\ Exponent = 0$

$\uparrow$
 Mantissa may be anything

4) Zero { Underflow }

Sign	Bias Exponent	Mantissa
1 Bit	8 bits	23 bits

$S = 0$
 $B.E. = 0$
 Mantissa = 00 ... 00

Examples on Floating Point Numbers

- ❑ **GATE 2022 CS** – Consider three floating point numbers A, B and C stored in registers Ra, Rb and Rc, respectively as per IEEE 754 single precision floating point format. The 32 bit constant stored in these registers are as follows:

$$\begin{aligned} R_a &= 0xC1400000 = \underline{1100\ 0001\ 0100\ 0\dots} = -2^3 \times 1.1 \\ R_b &= 0x42100000 = \underline{0100\ 0010\ 0001\ 0\dots} = +2^5 \times 1.001 \\ R_c &= 0x41400000 = \underline{0100\ 0001\ 0100\ 0\dots} = +2^3 \times 1.1 \end{aligned}$$

- ❑ Which one of the following is false?

- ✓ A. $A + C = 0$
✗ B. $C = A + B$
✓ C. $B = 3C$
✓ D. $(B - C) > 0$

$$A = -1100 = -12$$

$$B = \overset{32}{1}\overset{16}{0}\overset{8}{0}\overset{4}{0}\overset{2}{0} = 36$$

$$C = 1100 = 12$$

❑ **GATE 2019 CS** – Consider three floating point registers Ra, Rb and Rc that stores numbers as per IEEE 754 single precision floating point format. Assume that R1 and R2 contain the values 0x42200000 and 0xC1200000, respectively.

❑ If $R3 = R1 / R2$, what is the value stored in R3.

A. 0x40800000

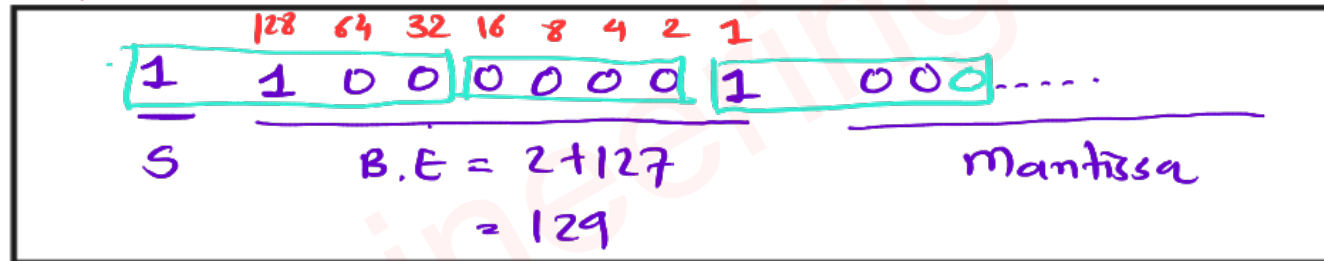
✓ B. 0xC0800000

C. 0x83400000

D. 0xC8500000

$$R_1 = \boxed{0} \boxed{100\ 0010\ 0010\ 0\dots} = + 2^5 \times 1.01$$
$$R_2 = \boxed{1} \boxed{100\ 0001\ 0010\ 0\dots} = - 2^3 \times 1.01$$

$$\rightarrow R_3 = R_1 / R_2 = - 1.00 \times 2^2$$



C0800000

Examples on Floating Point Numbers

- ❑ **GATE 2021 CS** – Consider the following representation of a number IEEE 754 Single Precision floating point format with a bias of 127.

$$S = \textcircled{1} -ve$$

$$E = \textcircled{1000\ 0001}$$

$$F = \text{1111\ 0000\ 0000\ 0000\ 0000\ 000}$$

- ❑ Here S, E and F denote sign, Exponent and Fraction Components of the floating point representation.
- ❑ The decimal value corresponding to the above representation is^{-7.75}

$$\begin{aligned} \Rightarrow T &= E - 127 \\ &= 129 - 127 \\ &= 2 \end{aligned}$$

$$\begin{aligned} \rightarrow \text{Number} &= -1.\underline{1111} \times 2^2 \\ &= -111.11 \\ &= -(1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2}) \\ &= -7.75 \end{aligned}$$

❑ **GATE 2021 CS** – The format of single precision floating point IEEE 754 standard is:



❑ Which of the following choices is correct with respect to the smallest normalized positive number represented using the standard?

- A. Exponent = 000 0000, Mantissa = 00 00
- B. Exponent = 000 0000, Mantissa = 00 01
- ✓ C. Exponent = 000 0001, Mantissa = 00 00
- D. Exponent = 000 0001, Mantissa = 00 01

$$= +1.00... \times 2^{-126}$$
$$= +1.00...01 \times 2^{-126}$$

$$E = 0000\ 0001$$

❑ **GATE 2008 CS** – In the IEEE floating point representation, the hexadecimal value 0x00000000 corresponds to

- A. The Normalized value 2^{-127}
- B. The Normalized value 2^{-126}
- C. The Normalized value 0
- D. ✓ The Special value 0

Engineering Funda

□ **GATE 2014 CS** – A decimal number is given by -14.25. Represent that in IEEE single precision format.

✓ A. 0xC1640000

B. 0x416C0000

C. 0x41640000

D. 0xC16C0000

$$\begin{array}{r} 0.25 \times 2 = \underline{0.5} \quad 0 \downarrow \\ 0.5 \times 2 = \underline{1.0} \quad 1 \downarrow \end{array}$$

→ Normalized Number = -1.11001×2^3 Exponent

Diagram illustrating the IEEE 754 single-precision floating-point format (32 bits):

- Sign (1 bit):** The most significant bit, labeled "Sign".
- Exponent (8 bits):** Labeled "bias exponent". The bits are 1, 1, 0, 0, 0, 0, 1, 0. Above the bits are weights: 128, 64, 32, 16, 8, 4, 2, 1.
- Mantissa (23 bits):** Labeled "mantissa (23 bits)". The bits are 1, 1, 0, 0, 1, 0.

$$= 3 + 127 = \underline{130}$$

❑ **GATE 2012 CS** – The decimal value 0.5 in IEEE single precision floating point representation has

- A. Fraction bits of 000 000 and exponent value of 0
- ✓ B. Fraction bits of 000 000 and exponent value of +126
- C. Fraction bits of 100 000 and exponent value of 0
- D. None

$$(0.5)_{10} = (0.1)_2$$
$$= + \underbrace{1.000}_\text{fraction} \times 2^{-1}$$

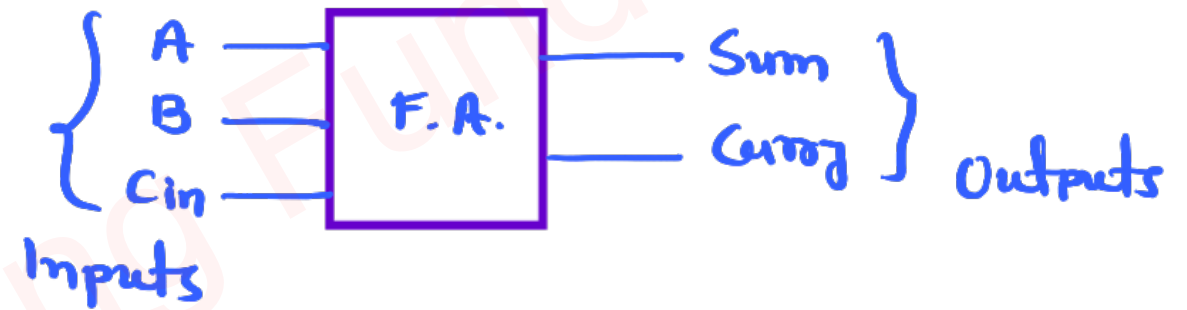
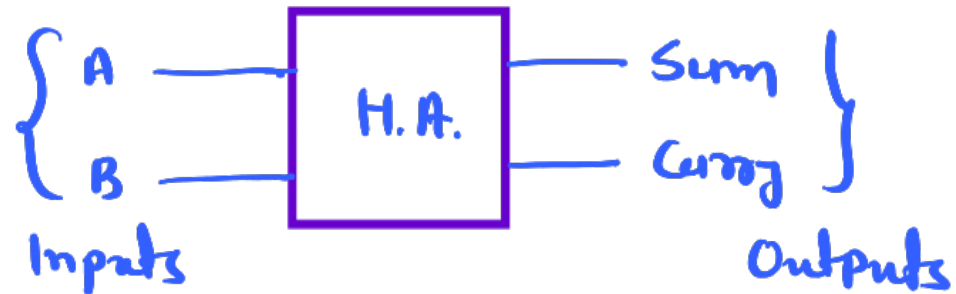
$$B.E = E + 127$$

$$= -1 + 127$$

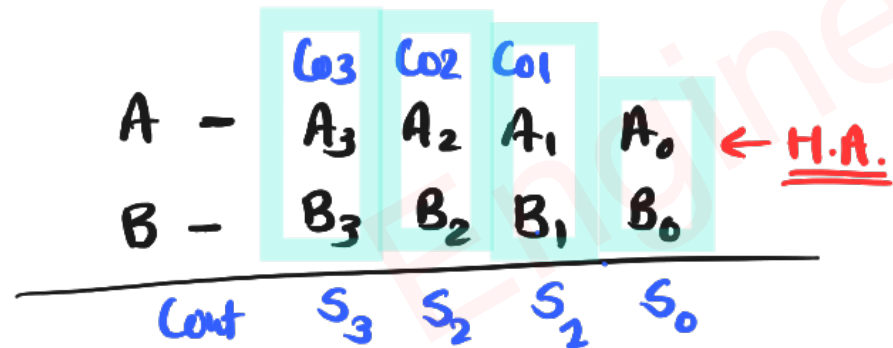
$$= 126$$

HALF Adder & FULL Adder

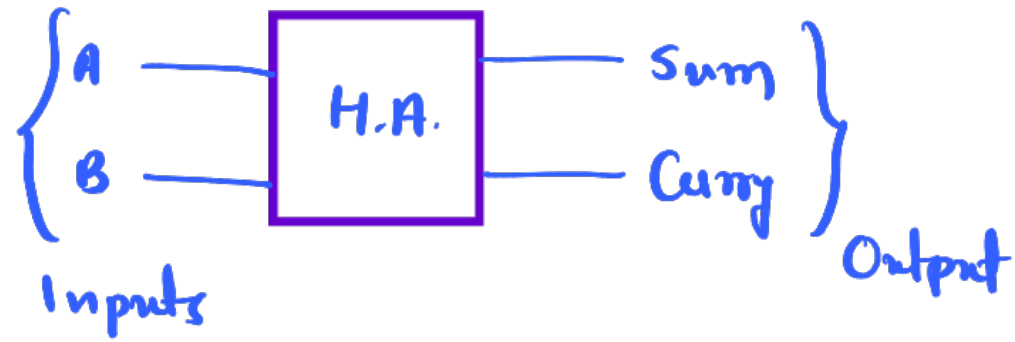
⇒ Basic Structure of Half Adder & Full Adder.



⇒ Multiple bits addition using H.A & F.A.

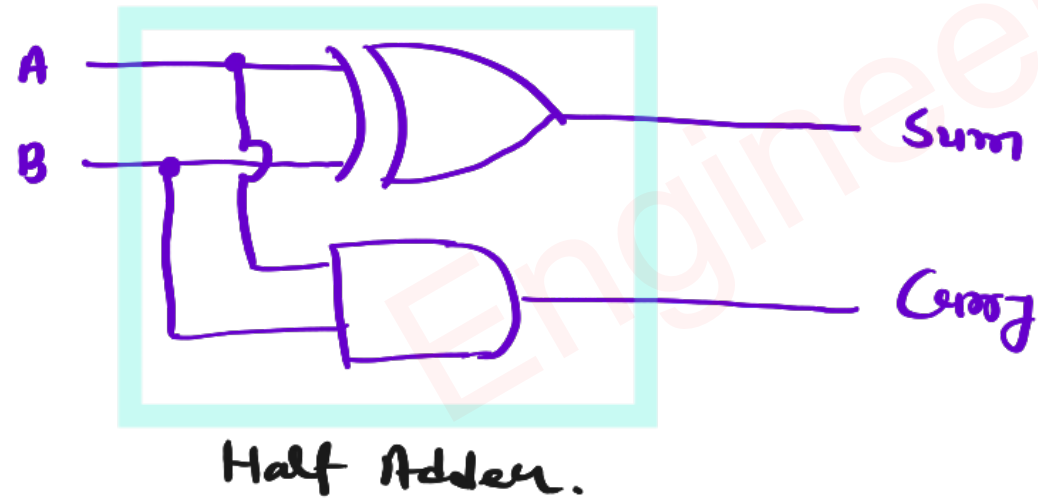


→ Half Adder

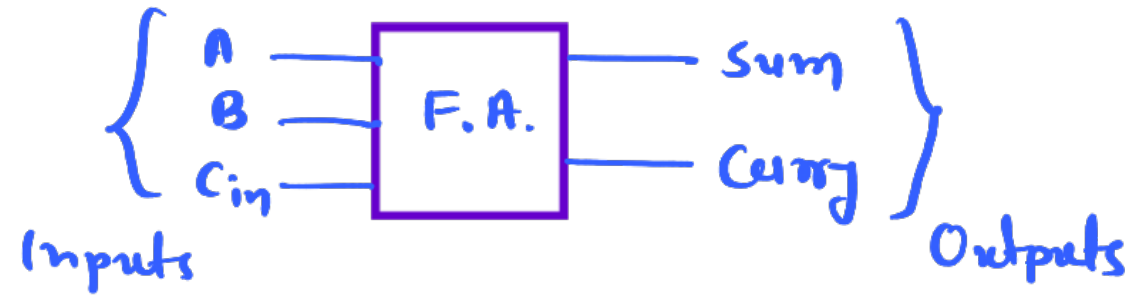


Input		Output	
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

→ $\text{Sum} = A \oplus B$ → $\text{Carry} = A \cdot B$



⇒ Full Adder



Truth table for Sum:

A	B	C_{in}	Sum			
			00	01	11	10
0	0	0	0	1	0	1
1	0	0	1	0	1	0

$$\text{Sum} = A \oplus B \oplus C_{in}$$

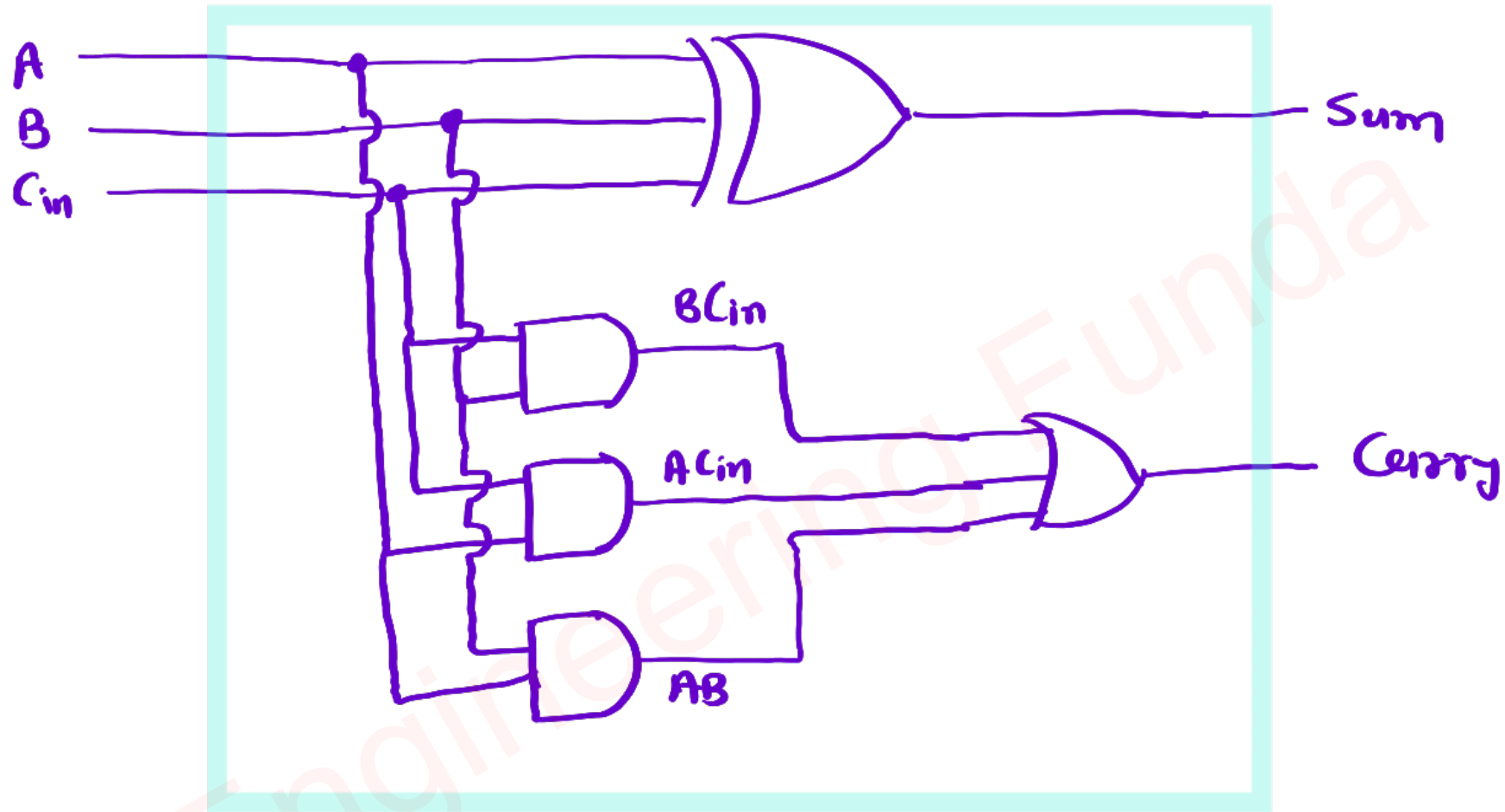
Truth table for Carry:

A	B	C_{in}	Carry			
			00	01	11	10
0	0	0	0	0	1	0
1	0	0	0	1	1	1

$$\text{Carry} = \underline{B C_{in}} + \underline{A C_{in}} + \underline{A B}$$

Complete Truth table for Full Adder:

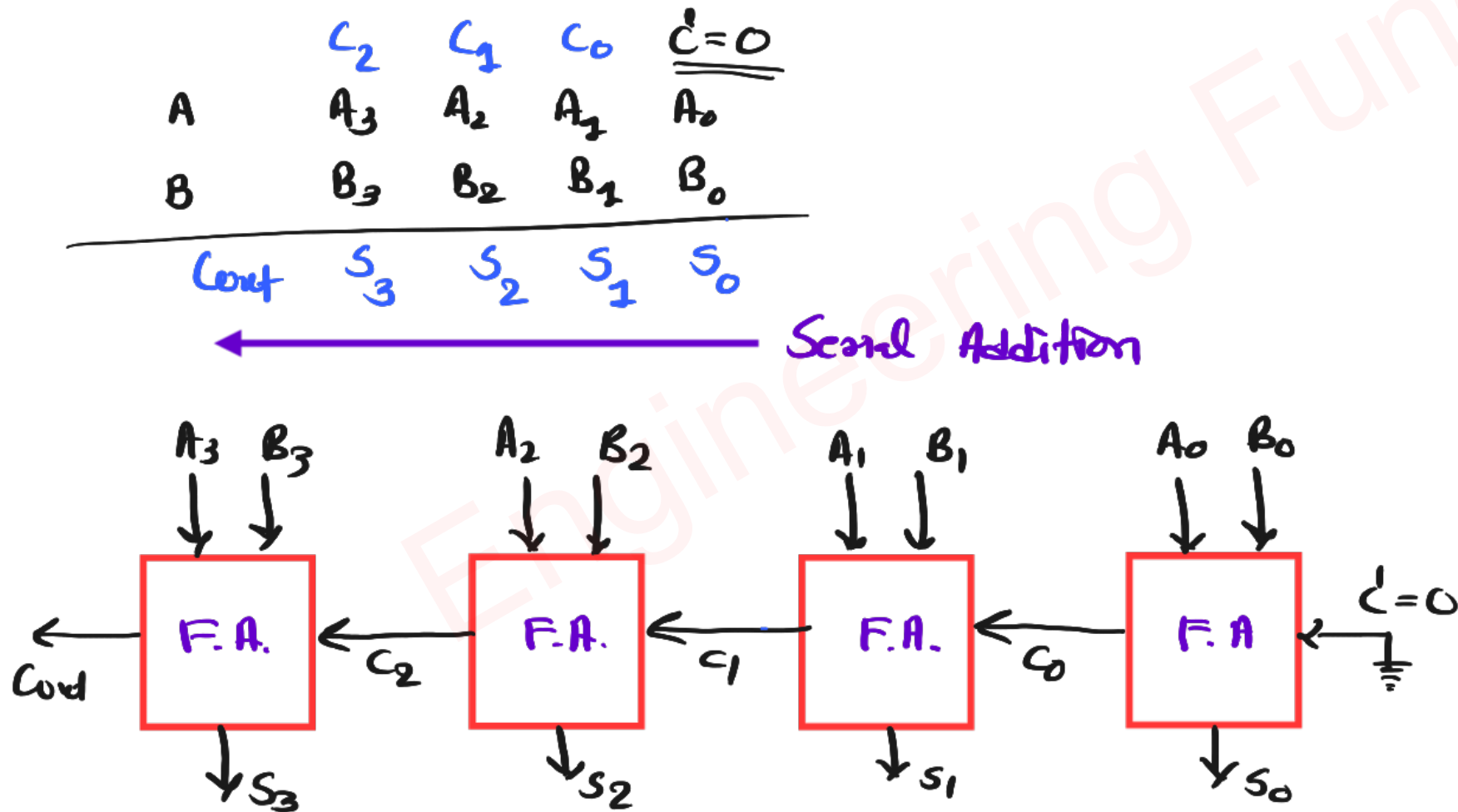
Inputs			Outputs	
A	B	C_{in}	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



Full Adder Circuit

Serial Adder for Multiple bits Addition, Ripple Carry Adder

→ 4 bits Addition



❖ Basics of Ripple Carry Adder

- ❑ Here, carry is rippled to next stage of addition. So, it is referred ripple carry adder.
- ❑ Here, Addition is forwarded serially. So, it is also referred serial carry adder.
- ❑ It takes n numbers of machine cycles to add n bits serially. So, it is slow adder.

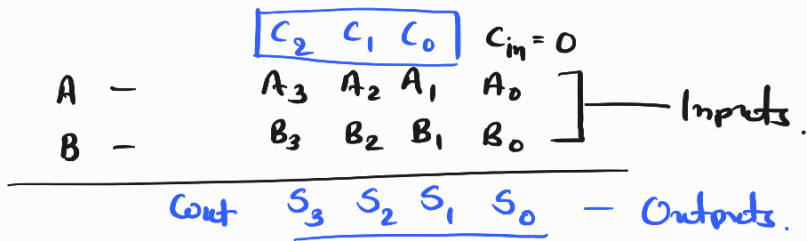
Carry Look Ahead Adder - CLA

Parallel Adder

❖ Basics of Carry Look Ahead Adder - CLA

- ☐ CLA adds all the bits simultaneously.
- ☐ If we “Predict” or “Look Ahead” carry, then only simultaneous addition is possible.
- ☐ CLA adds carry with independent on numbers of bits to be added. So it is very fast compared to serial adder.

→ 4 bits Addition



$$\begin{aligned}
 \rightarrow \text{Carry} &= AB + AC_{in} + BC_{in} \\
 &= AB + C_{in}(A+B) \\
 &= G + C_{in}P
 \end{aligned}$$

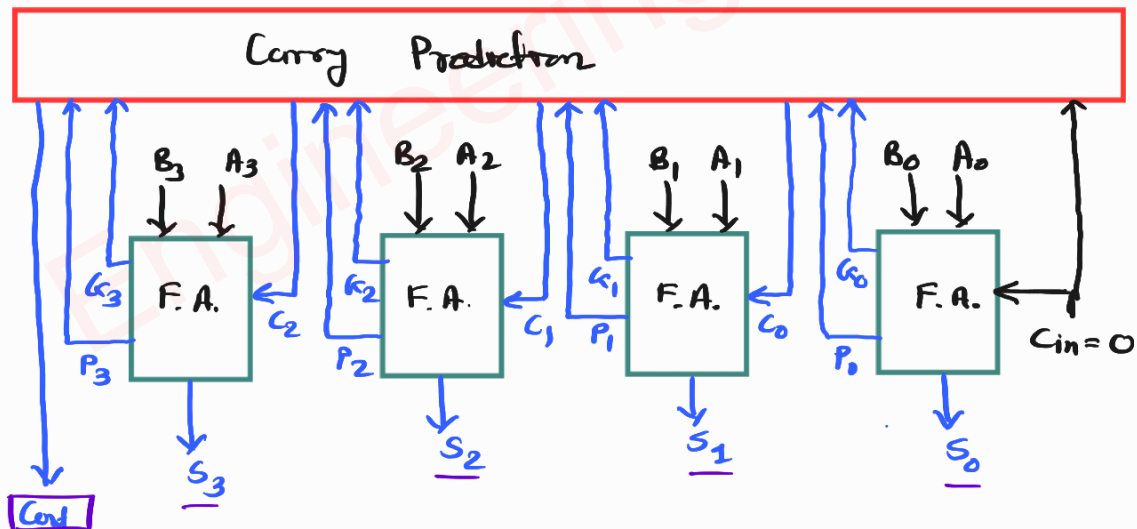
$$\rightarrow C_{in} = 0$$

$$\rightarrow C_0 = G_0 + C_{in}P_0$$

$$\begin{aligned}
 \rightarrow C_1 &= G_1 + C_0P_1 = G_1 + (G_0 + C_{in}P_0)P_1 \\
 &= G_1 + G_0P_1 + C_{in}P_0P_1
 \end{aligned}$$

$$\begin{aligned}
 \rightarrow C_2 &= G_2 + C_1P_2 = G_2 + (G_1 + G_0P_1 + C_{in}P_0P_1)P_2 \\
 &= G_2 + G_1P_2 + G_0P_1P_2 + C_{in}P_0P_1P_2
 \end{aligned}$$

$$\begin{aligned}
 \rightarrow Cont &= G_3 + C_2P_3 = G_3 + (G_2 + G_1P_2 + G_0P_1P_2 + C_{in}P_0P_1P_2)P_3 \\
 &= G_3 + G_2P_3 + G_1P_2P_3 + G_0P_1P_2P_3 + C_{in}P_0P_1P_2P_3
 \end{aligned}$$

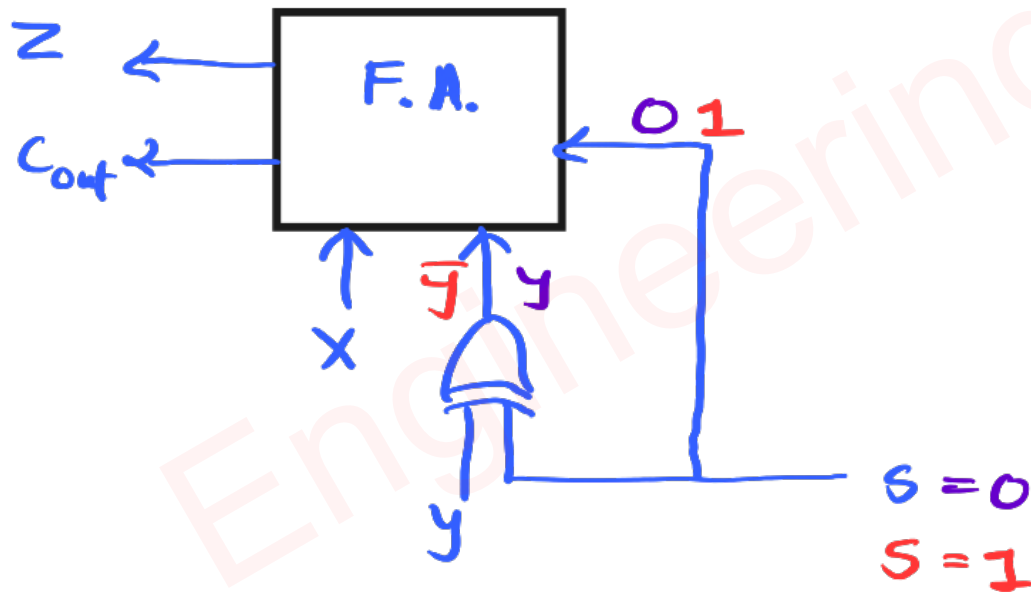


Subtractor in Computer

$$\begin{aligned}
 \Rightarrow Z &= x - y \\
 &= x + (2's \text{ Comp. of } y) \\
 &= \underline{x} + \underline{\bar{y}} + \underline{1}
 \end{aligned}$$

s	y	
0	0	0
0	1	1
1	0	1
1	1	0

$\left. \begin{matrix} 0 \\ 0 \end{matrix} \right\} = y$
 $\left. \begin{matrix} 1 \\ 1 \end{matrix} \right\} = \bar{y}$



$$\Rightarrow s = 0, \quad Z = x + y + 0 = x + y$$

$$\Rightarrow s = 1, \quad Z = x + \bar{y} + 1$$

4 Bits Adder Subtractor

⇒ A & B are of 4 bits.

$$\Rightarrow \underline{Z = A + B}$$
$$C = 0$$

$$\begin{aligned}\Rightarrow Z &= A - B \\ &= A + (2^i \text{ Comp. of } B) \\ &= A + \overline{B} + 1\end{aligned}$$

$$C = 1$$

C	B	y
0	0	0
0	1	1
1	0	1
1	1	0

$y = B$ (for $C=0$)
 $y = \overline{B}$ (for $C=1$)

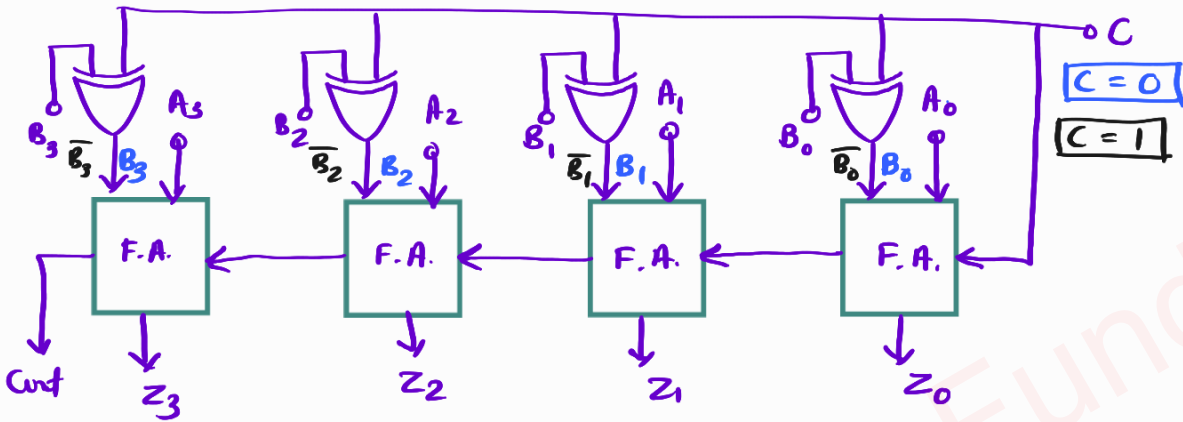


Diagram illustrating a 3-stage shift register with feedback. The stages are labeled F.A. (Feedback Accumulator). The outputs are labeled z_0 , z_1 , and z_2 . The output z_2 is fed back into the input of the first stage.

Booth's Algorithm Example

Multiply $5 \times (-6)$ using Booth's Algorithm

$$\begin{aligned} \leadsto +5 &= 0101 & \leadsto +6 &= 0110 & \leftarrow \\ & & -6 &= 1010 \end{aligned}$$

Operation	Accumulator A	Multiplicand Q $Q_3 \ Q_2 \ Q_1 \ Q_0$	Q_{-1}	Multiplicand m
	0 0 0 0	0 1 0 1 0		$m = 1010$ $-m = 0110$
$Q_0Q_{-1} = 10$ Sub m Right Shift	$\begin{array}{r} 0 \ 1 \ 1 \ 0 \\ 0 \ 1 \ 1 \ 0 \\ \hline 0 \ 0 \ 1 \ 1 \end{array}$	0 1 0 1 0 0 1 0 1		
$Q_0Q_{-1} = 01$ Add m Right Shift	$\begin{array}{r} 1 \ 0 \ 1 \ 0 \\ 1 \ 1 \ 0 \ 1 \\ \hline 1 \ 1 \ 1 \ 0 \end{array}$	0 0 1 0 1 0 0 1 0		
$Q_0Q_{-1} = 10$ Sub m Right Shift	$\begin{array}{r} 0 \ 1 \ 1 \ 0 \\ 0 \ 1 \ 0 \ 0 \\ \hline 0 \ 0 \ 1 \ 0 \end{array}$	1 0 0 1 0 1 0 0 1		
$Q_0Q_{-1} = 01$ Add m Right Shift	$\begin{array}{r} 1 \ 0 \ 1 \ 0 \\ 1 \ 1 \ 0 \ 0 \\ \hline 1 \ 1 \ 1 \ 0 \end{array}$	0 1 0 0 1 1 1 0 0 0 1 0	0	

$$\text{Ans} = 11100010 = \boxed{-30}$$

$$\downarrow$$

$$\text{Sign} = -ve$$

$$= 00011110$$

16 8 4 2 1

Booth's Algorithm Example

Multiply 4×6 using Booth's Algorithm.

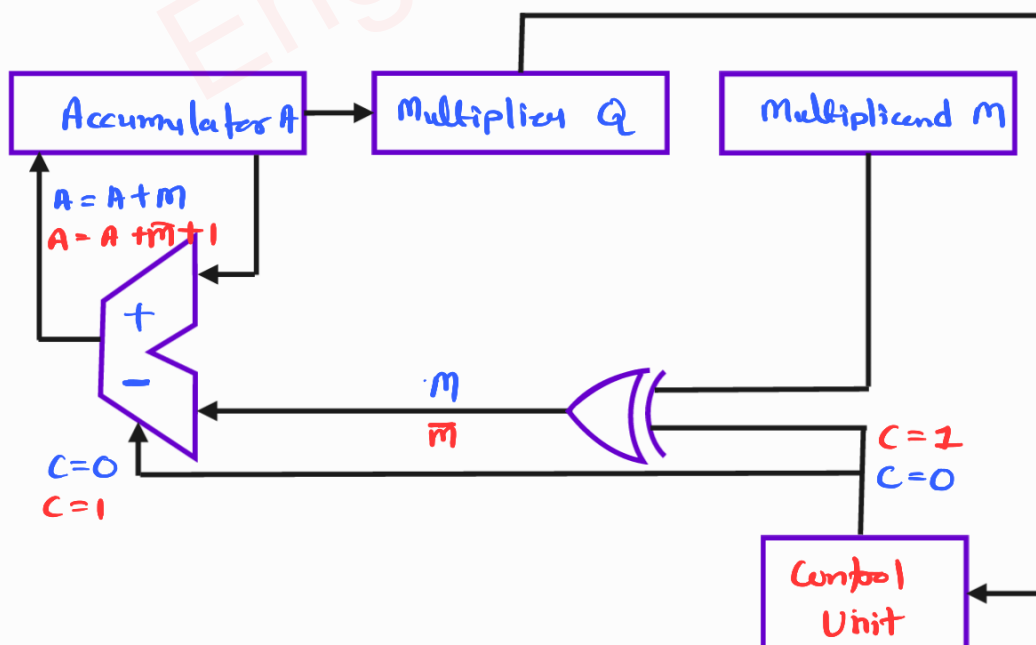
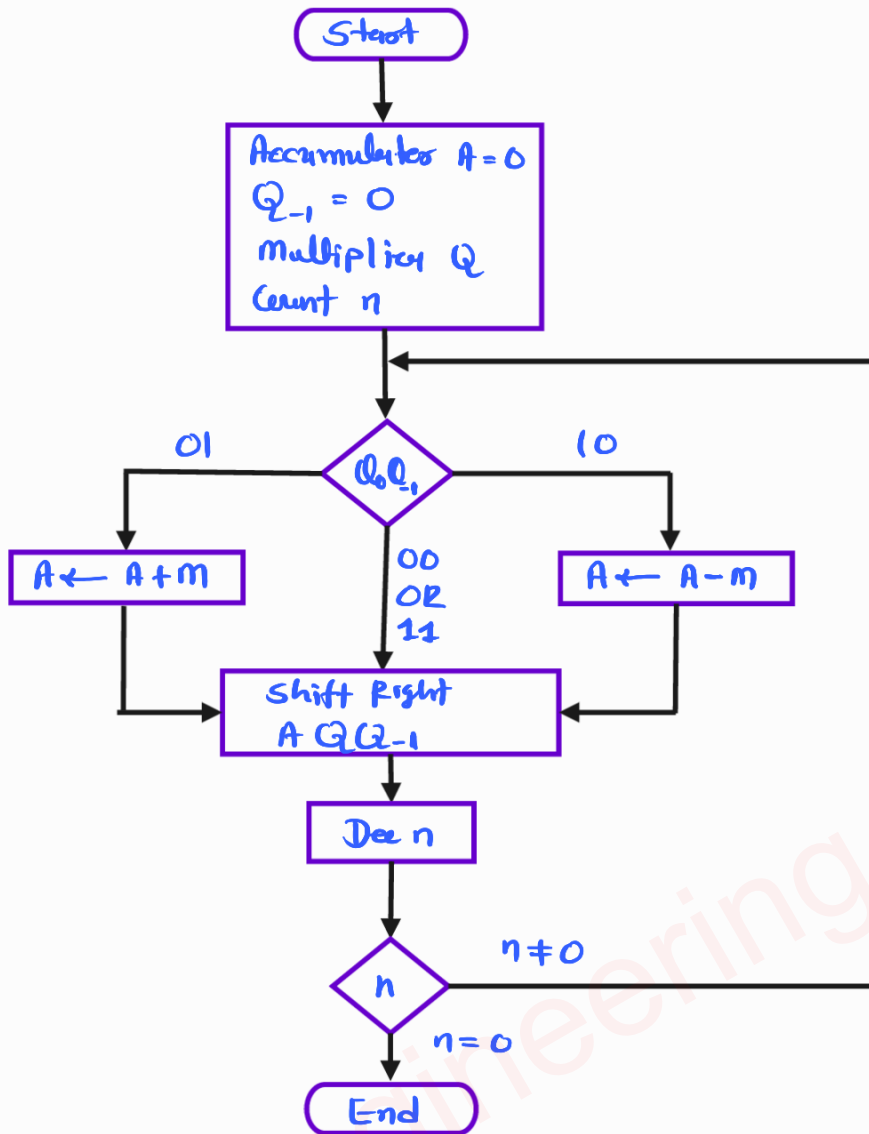
$$\begin{aligned} \rightarrow +4 &= 0100 & \rightarrow +6 &= 0110 \\ & & -6 &= 1010 \end{aligned}$$

Operation	Accumulator A	Multiplicand Q $Q_3 \ Q_2 \ Q_1 \ Q_0$	Q_{-1}	Multiplicand m $m = 0110$ $-m = 1010$
.	0 0 0 0	0 1 0 0	0	
$Q_0 Q_{-1} = 00$ Right shift (1)	0 0 0 0	0 1 0 0	0	
$Q_0 Q_{-1} = 00$ Right shift (2)	0 0 0 0	0 0 1 0	0	
$Q_0 Q_{-1} = 10$ Sub m Right shift (3)	1 0 1 0	0 0 0 1	0	
$Q_0 Q_{-1} = 01$ Add m Right shift (4)	0 1 1 0	0 0 0 0	0	

$$\text{Ans} = \overset{16 \ 8 \ 4 \ 2 \ 1}{00011000} = \underline{\underline{+24}}$$

↑
+ve

Booth's Algorithm



Restoring Division

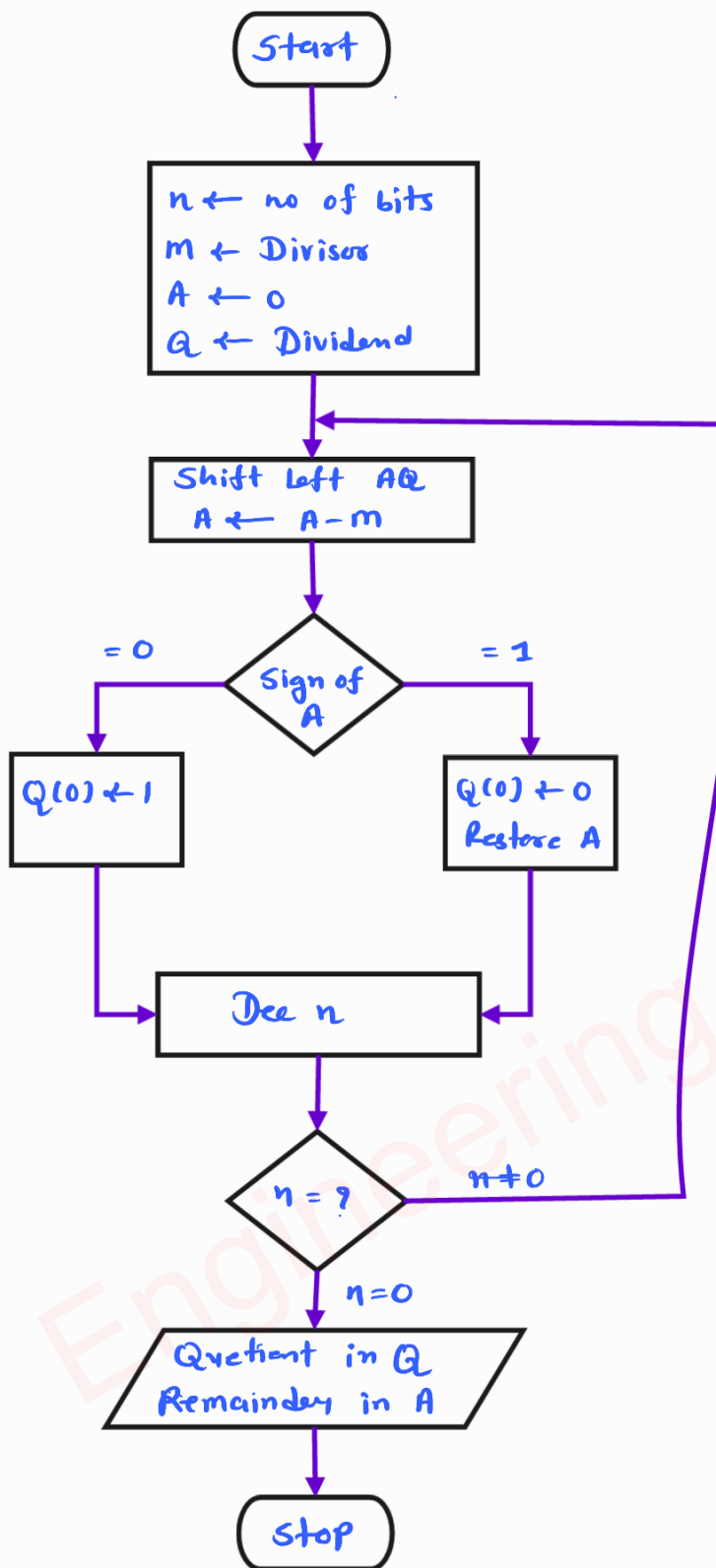
Perform $14 \div 3$ division using restoring division.

$\overbrace{14}^{\text{Divident } Q}$
 $\overbrace{3}^{\text{Divisor } m}$
 $Q = 1110$ [n = 4 bits]

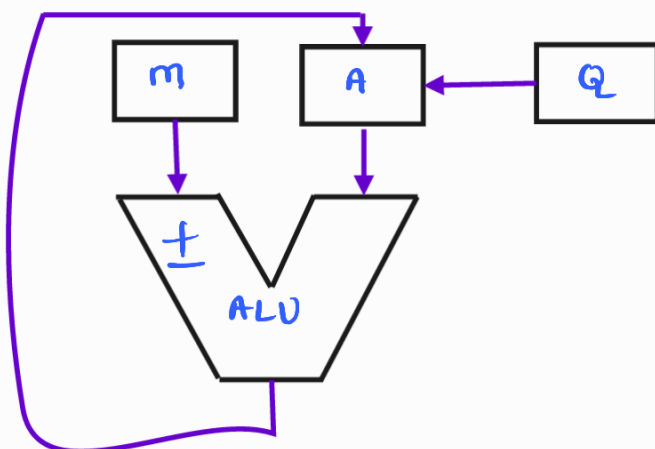
Operation	Accumulator A	Divident Q	Divisor m
Initial values	0 0 0 0 0	1 1 1 0	m = 00011 ←
Shift left	0 0 0 0 1	1 1 0 _	-m = 11101
Sub m	1 1 1 0 1		
	<u>1 1 1 1 0</u>		
Restore (Add m)	0 0 0 0 1	1 1 0 0	
Shift left	0 0 0 1 1	1 0 0 _	
Sub m	1 1 1 0 1		
	<u>0 0 0 0 0</u>	1 0 0 1	
Shift left	0 0 0 0 1	0 0 1 _	
Sub m	1 1 1 0 1		
	<u>1 1 1 1 0</u>		
Restore (Add m)	0 0 0 0 1	0 0 1 0	
Shift left	0 0 0 1 0	0 1 0 _	
Sub m	1 1 1 0 1		
	<u>1 1 1 1 1</u>		
Restore (Add m)	0 0 0 1 0	0 1 0 0	

A = 2
Remainder

Q = 4
Quotient



⇒ Circuit of Restoring division



Non Restoring Division

Perform $14 \div 3$ division using Non restoring division.

Divident Q Divisor m

$Q = 1110$ ($n = 4$ bits)

Operation	Accumulator A	Divident Q	Divisor m
Initialization	0 0 0 0 0	1 1 1 0	$m = 00011 \leftarrow$
Shift Left Sub m	0 0 0 0 1 1 1 1 0 1 1 1 1 0	1 1 0 -	$-m = 11101$
Shift Left Add m	1 1 1 0 1 0 0 0 1 1 0 0 0 0 0	1 0 0 -	
Shift Left Sub m	0 0 0 0 1 1 1 1 0 1 1 1 1 1 0	0 0 1 -	
Shift Left Add m	1 1 1 0 0 0 0 0 1 1 1 1 1 1 1	0 1 0 -	

Restore

If A is
-ve in
last step.

$\rightarrow$ 0 0 0 1 0 0 1 0 0
Ans $A = 2$ $Q = 4$
 Remainder Quotient

